

Recruitly · Mini Hiring Pipeline

A small applicant tracking portal: a recruiter moves candidates through Applied → Screening → Interview → Offer → Hired, with a history that can never be altered, and finds the people she needs with a typo-tolerant search and plain filters.

GitHub repo

<https://github.com/Shailly0502/Recruitly>

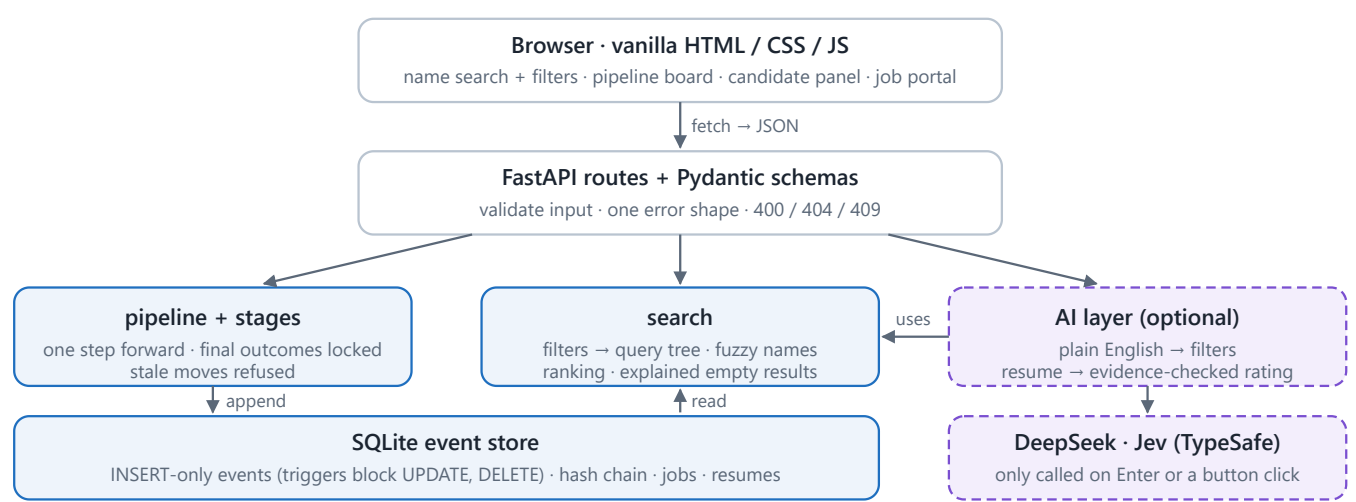
README

[How to run, decisions and why, what I'd do with more time, where I disagreed with the AI](#)

Full design

[docs/ARCHITECTURE.md](#)

Architecture



The core (stages, pipeline, store, search) is plain Python with no web framework and no knowledge of the AI layer. The app runs fully without any API keys.

History that can't be changed

- Every action is an **event**. A candidate's stage and time in stage are computed from events, so there's no status column to drift.
- SQLite triggers** block `UPDATE` and `DELETE`, so even code that bypasses the API can't rewrite history.
- Each event stores the previous event's **hash**, so hand-editing the database file is detected by `/api/audit/verify`.
- Moves send the stage the screen showed. A stale move (for example a double click) is refused rather than skipping a stage.

Search: filters, not syntax

- A **name box** with typo tolerance (optimal string alignment, so "sharam" is 1 edit from "sharma"), plus **filters**: current stage, excluded stages, time in stage, reached a stage since / before / on a day, never reached, and job.
- Filters become a small **query tree** that matches, **ranks** (best name first, then recency or longest wait) and **explains** empty results in plain English.
- A line above the results always says what's being searched: "Showing candidates who..."

The brief's questions

RECRUITER ASKS	HOW	RECRUITER ASKS	HOW
Find Priya Sharma, typed "sharam"	Name box	Reached Offer but not hired	Reached: Offer + Exclude: Hired
Who's in Interview right now?	Stage: Interview	Everyone except rejected	Exclude: Rejected
Stuck in Screening > a week	Screening + More than a week	Combine and rank	All filters AND together; best match first
Moved to Interview since Monday	Reached: Interview, Since Monday	Doesn't make sense	Told why, never a bare empty list

AI layer optional

Ask in plain English

- Runs only on Enter, and only if the typed words match no name.
- **Jev** decides what the input is (name, question, off-topic). **DeepSeek** translates the question into a query.
- The **parser validates** it (one retry with the error fed back), and it's converted into the **same filters** a recruiter would choose. Anything no menu can hold (for example OR across filters) runs as its own plain-English chip.
- Code **describes the filters back in English** and Jev checks that the description answers the question. Confident: run it. Unsure: offer it as a suggestion.
- Only the question text is sent, never candidate records.

Candidate rating, as advice only

- Runs only when the recruiter clicks **Create candidate rating**.
- DeepSeek extracts skills, roles and experience. Each fact must come with a **quote that really appears in the resume**, or it's dropped.
- Jev scores skills, experience and relevance against written rubrics, and sees **no name or personal details**. Code scores budget fit and the overall: $2 \times (0.35 \cdot \text{skills} + 0.30 \cdot \text{exp} + 0.20 \cdot \text{relevance} + 0.15 \cdot \text{budget})$.
- Low-confidence scores are flagged "needs review". A rating **never moves a candidate**, and each one is saved to the history.

Evaluation

- **Search eval:** 30 plain-English questions. A translation is correct only if its filters return the same candidates as the expected answer.
- **Rating eval:** agreement within 1 point, **consistency** (same resume, same score twice) and **fairness** (same resume under a different name gets the same score).

Where I disagreed with the AI

How it evaluates a candidate. The AI's plan was to score candidates by matching extracted skills, years and roles against the job description with a rubric, and to do it automatically for everyone who applied. Keyword and years overlap is a proxy, not a measure of a good candidate: it misses growth, impact, transferable skills and team fit, and it can put a keyword-heavy resume above a better person. Recognising a good candidate takes time and training on real hiring outcomes. A number a recruiter has to double-check adds manual work instead of removing it.

What changed: ratings run only when the recruiter clicks a button, they're advice that never moves anyone, every fact shows its evidence, and a properly trained scoring model is first on my list below.

What I'd do with more time

- **A proper ATS scoring model** trained on real outcomes and recruiter feedback, with regular bias audits.
- **A better database:** a normalised schema with foreign keys (candidates, jobs, applications, events), PostgreSQL with migrations, and filtering in SQL.
- **UI and UX:** drag-and-drop board, bulk actions, saved searches, notes and tags, interview scheduling, email templates.
- **A recruiter playground:** a sandbox with synthetic candidates to try searches, rubrics and weights without touching real data.
- **Analytics and alerts:** time to hire, conversion at each stage, and "5 candidates stuck in Screening".
- Semantic search over resumes, AI-drafted interview questions, duplicate detection, user accounts with per-user audit.

Tech stack and running it

Python, FastAPI, Pydantic and SQLite on the server. Vanilla HTML, CSS and JavaScript with no build step. pypdfium2 for resumes. DeepSeek and TypeSafe Jev are optional.

```
python -m venv .venv → pip install -r requirements.txt → uvicorn app.main:app, then open localhost:8000
```

and click **Load demo candidates**.